# Übungsblatt 1: Bare-Metal, Polling und Interrupts

**Echtzeitsysteme (EZS)**

Prof. Dr.-Ing. Daniel Klünder

## Zielsetzung

In dieser ersten Laborübung machen Sie sich mit der Entwicklungsumgebung und der Hardware vertraut. Sie lernen Grenzen und Einsatzbedingungen von Polling-Architekturen (Super-Loops) kennen — insbesondere das *Heavy-Load-Problem* bei langen unzerlegten Abschnitten. Anschließend verlagern Sie die periodische LED-Aktion in einen Timer-Interrupt (Foreground/Background). Hardware: **Board NUCLEO-H755ZI-Q** (MB1363), **MCU STM32H755ZIT6**, Arbeit auf dem Cortex-M4.

---

## Aufgabe 1: Hello World (LED Blink)

Um die Toolchain zu überprüfen, erstellen wir ein einfaches Bare-Metal-Programm auf dem Cortex-M4-Kern.

1. Legen Sie ein neues Projekt für das Board **NUCLEO-H755ZI-Q** (MCU STM32H755ZIT6) in der STM32CubeIDE an.
2. Konfigurieren Sie die gelbe LED **LD2 an PE1** (MB1363) als `GPIO_Output` und weisen Sie sie dem Cortex-M4 zu (*Pin Context Assignment*). Boardrevision/Bestückung bei Bedarf in UM2408 prüfen.
3. Generieren Sie den Code; prüfen Sie GPIO- und Clock-Initialisierung.
4. Lassen Sie die LED in der `main.c` des CM4-Projekts toggeln (`HAL_Delay(500)`). Hinweis: Toggle alle 500 ms ergibt Flankenabstand 500 ms, aber einen vollständigen On-Off-Zyklus von $\approx 1\,\text{s}$ (1 Hz).

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

---

## Aufgabe 2: Das Polling-Desaster

Wir provozieren einen Timing-Fehler, der Grenzen einer Super-Loop bei langer Hintergrundarbeit zeigt. Polling ist nicht grundsätzlich ungeeignet — lange unzerlegte Arbeit kann aber das Zeitbudget sprengen.

1. Schreiben Sie eine *Heavy-Load*-Funktion (Zählschleife), deren Arbeit der Compiler nicht entfernt (Parameter/`volatile`, Ergebnis ggf. nach außen nutzen). Iterationen sind **keine** bekannte Millisekunden-Dauer; Buildoptionen notieren.
2. Rufen Sie die Funktion in der vorhandenen `while(1)` direkt vor dem LED-Toggle auf.
3. Beobachten Sie die LED. Unterscheiden Sie: **konstante** Last verlängert den Abstand konstant (noch kein Periodenjitter); für Schwankungen bewusst **variable** Lasten einsetzen.

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

---

## Aufgabe 3: Rettung durch den Timer

Wir stellen periodische Flanken per Hardware-Timer wieder her — unter dokumentierten Bedingungen messbar, aber ohne automatischen harten Fristnachweis.

1. Aktivieren Sie für den CM4 im Device Configuration Tool **TIM2**, *Clock Source* „Internal Clock“.
2. NVIC: Haken bei „TIM2 global interrupt“.
3. Berechnen Sie `PSC` und `ARR` für Update alle 500 ms ($f_{\mathrm{UPD}} = 2\,\mathrm{Hz}$):

$$f_{\mathrm{UPD}} = \frac{f_{\mathrm{TIM}}}{(PSC + 1)\,(ARR + 1)}$$

$f_{\mathrm{TIM}}$ aus dem Clock-Tree lesen (APB-Prescaler/TIMPRE beachten) — nicht automatisch `SystemCoreClock`.

4. Generieren Sie den Code. Vor der Schleife: `HAL_TIM_Base_Start_IT(&htim2)` mit Fehlerprüfung. Handler-Kette prüfen: `TIM2_IRQHandler` → `HAL_TIM_IRQHandler` → Callback.
5. Toggle/`HAL_Delay` aus der Main-Schleife entfernen (dort z. B. nur `Heavy_Load()`). Callback **einmal** im vorgesehenen USER-Block (nicht doppelt definieren).

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

---

## Aufgabe 4 (Bonus): Drei Messungen am Oszilloskop

Timing wird gemessen, nicht nur behauptet. Pin z. B. **PA0** erst nach Pinbelegung freigeben.

1. **Messung A:** Puls um `Heavy_Load()` (HIGH → Last → LOW). Breite = verstrichene Dauer inkl. möglicher IRQ-Unterbrechungen.
2. **Messung B:** periodische Flanken Main (Aufgabe 2) vs. TIM2-Callback (Aufgabe 3); gleiche Sollperiode; konstante vs. variable Last getrennt untersuchen.
3. **Messung C (optional):** Referenz/Stimulus mit gemeinsamem Zeitbezug vs. früher Handler-Marker; Anfang/Ende nennen (HAL-Callback ≠ reine Hardware-Entry).
4. Trigger, Zeitbasis, Sampleanzahl notieren. Schema: Min/Max/Mittel/Peak-to-Peak — Zahlen erst nach Messung. Formulierung: „gemessene Schwankung unter dieser Konfiguration innerhalb …“, nicht „Jitter ≈ 0“.

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*